

Energy-Aware Optimization of the QMR Routing Protocol for Flying Ad Hoc Networks

June 15, 2026

Author : Elkilani Narmin

Abstract

Flying Ad Hoc Networks (FANETs) are composed of cooperating UAVs that communicate without fixed infrastructure. Due to the high mobility of UAVs and the dynamic network topology, routing remains a major challenge. While traditional routing protocols often struggle to adapt to changing conditions, reinforcement learning based approaches offer greater adaptability. This paper focuses on QMR, a multi-objective Q-learning routing protocol that considers both delay and residual energy. However, QMR uses a fixed reward weight that consistently favors delay and a uniform Q-table initialization that ignores energy information. To address these limitations, we introduce a dynamic reward weight based on the forwarding UAV's battery level and an energy-aware Q-table initialization strategy. A comparison on a small illustrative topology shows that the proposed approach consistently selects more energy-efficient relays, resulting in longer path lifetimes. Further simulations on static FANET topologies of up to 49 UAVs demonstrate improvements in average bottleneck path lifetime compared to the original protocol, particularly for networks with more than five UAVs. These results highlight the potential of energy-aware reinforcement learning for extending network lifetime in FANETs.

1 Introduction

Unmanned Aerial Vehicles (UAVs) have attracted significant attention in recent years due to their flexibility, rapid deployment capabilities, and wide range of applications. When multiple UAVs cooperate and communicate without relying on fixed infrastructure, they form a Flying Ad Hoc Network (FANET). FANETs have become an important component of next-generation wireless networks and the Internet of Everything, enabling applications such as environmental monitoring, disaster management, surveillance, search-and-rescue operations, and emergency communications. Compared with traditional Mobile Ad Hoc Networks (MANETs), FANETs exhibit higher node mobility, more frequent topology changes, and stricter constraints on communication reliability and energy consumption. One of the main challenges in FANETs is the design of efficient routing protocols capable of maintaining reliable communication despite the highly dynamic network topology. Routing decisions must not only ensure timely packet delivery but also account for the limited battery capacity of UAVs, as excessive energy consumption can significantly reduce network lifetime. Consequently, achieving a balance between communication performance and energy efficiency remains a critical research issue in FANET routing.

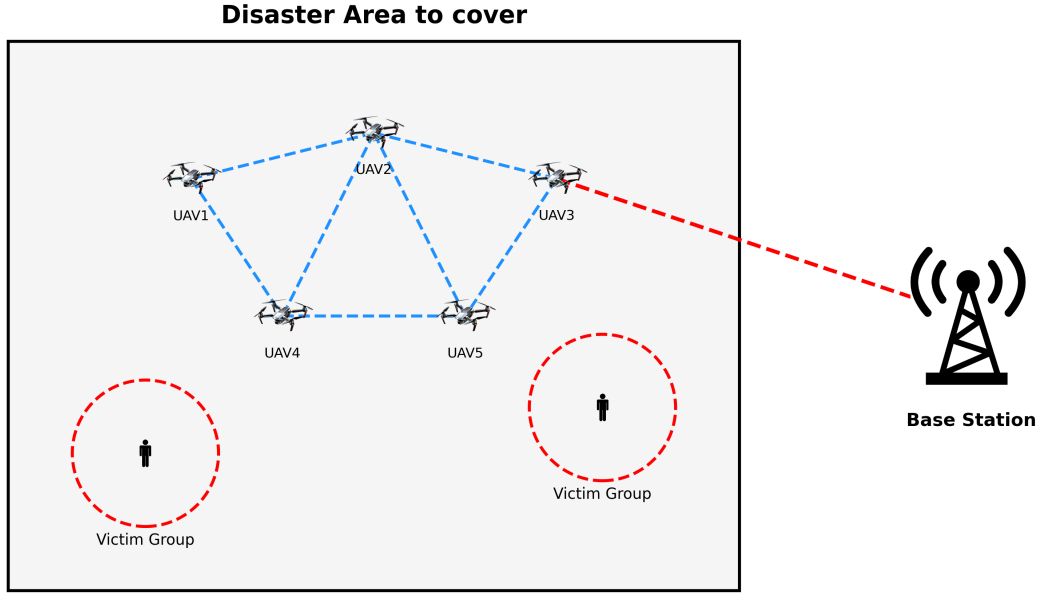


Figure 1: Disaster scenario

2 Context

Consider a search-and-rescue operation in a remote disaster area where communication infrastructure is unavailable, as shown in Figure 1. Rescue teams and injured victims need to exchange information, but the terrain and lack of network coverage make direct communication impossible. A practical solution is to deploy a fleet of UAVs that forms a Flying Ad Hoc Network (FANET). The UAVs act as relay nodes, ensuring connectivity between the rescue teams and a Base station while providing coverage of the entire affected area. Since UAVs are powered by limited batteries, maximizing network lifetime is crucial. Routing decisions should therefore not only guarantee reliable communication but also use the available energy efficiently to maintain network connectivity for as long as possible. This work investigates routing mechanisms to address this challenge.

Contents

1	Introduction	1
2	Context	2
3	Problem Statement	4
4	Related Work	4
4.1	Classical routing protocols	4
4.2	AI enabled routing protocols	7
5	Protocol to Modify : QMR	11
5.1	QMR Reinforcement learning algorithm	11
5.2	Algorithm steps	12
6	Protocol Modification: Modifying the Reward function	13
6.1	Reward function	13
6.1.1	First Protocol Optimization: Dynamic ω	14
7	Experiments	14
7.1	Initial Implementation Corrections and Adaptations	15
7.2	First Experiment : Paths Comparison	16
7.2.1	Start of the experiment: Why Fixed and Dynamic ω Initially Behave Identically	16
7.2.2	Second protocol optimization: Energy-Based Initial Q-Values	17
7.2.3	Results	17
7.3	Second Experiment: Network simulations	22
7.3.1	Experiment Parameters	22
7.3.2	Results	23
8	Perspectives	23
9	Conclusion	24

3 Problem Statement

How can reliable communication be maintained in a FANET while maximizing network lifetime?

4 Related Work

4.1 Classical routing protocols

In this section, we briefly review routing protocols that were mainly introduced for low-speed ad hoc networks. These routing protocols do not adapt to high mobility and abrupt changes we find in UAV networks. Therefore, most of them are not applicable for high-speed UAV applications.

1. *Proactive routing* : Proactive routing protocols attempt to keep an up to-date topological map of the entire network. With this map, the route is known and immediately available when a packet needs to be sent. The approach is similar to the one used in wired IP networks. [1] It is not adapted for UAV networks, thus, it will not be cited in the Routing Protocols table.
2. *Reactive Routing*: In contrast to proactive routing, reactive routing does not attempt to continuously determine the network connectivity. Instead, a route determination procedure is invoked on demand when a packet needs to be forwarded. [1]
3. *Hybrid routing*: Hybrid routing protocols combine proactive and reactive routing features. The route is initially determined with a proactive protocol. However, a reactive routing protocol is activated when a substantial network topology is recognized or a previously established route breaks. [2]
4. *Position-based Routing* Position-based routing protocols find the optimal route based on location information. For example, the next node can be selected based on its distance to the current node or to the destination. [2]
5. *Hierarchical Routing* Hierarchical routing protocols consider nodes arranged hierarchically, where the lower layers can form clusters. Each node typically holds information only about its neighbors, stored in a table that is updated through hello packets. Each cluster head communicates with the rest of the cluster heads to select the best path. [2]
6. *Probabilistic Routing* Probabilistic routing protocols find multiple routes from source to destination, which can be selected based on probabilistic mechanisms to cope with network congestion and security. [2]

Table 1: Review of Classical Routing protocols for UAVs

Routing Protocol	Type	Central /Distributed	Definition	Key Characteristics	Limitations
Dynamic Source Routing (DSR) [3]	Reactive	Distributed	• Uses source routing where the full path is included in the packet header and stored in a route cache. • Initiates route discovery by broadcasting Route Request (RREQ) packets when no route is known. • Intermediate nodes rebroadcast RREQ packets until the destination is reached or a route is found.	• <i>Low Control Overhead:</i> Does not require global periodic routing advertisements. • <i>Route Caching:</i> Route caching reduces future delay and the control overhead.	• <i>Large Header Size:</i> Large packet header size due to full path information. • <i>High Setup Delay:</i> High connection setup delay due to route discovery. • <i>Stale Routes:</i> Stale cached routes could result in inconsistencies. • <i>Scalability:</i> Poor scalability in large networks due to long route headers and flooding overhead.
Ad hoc On-Demand Distance Vector (AODV)	Reactive	Distributed	• Broadcasts RREQ packets to find routes when needed. • Nodes reply with RREP if a valid route exists. • Routes are established using reverse paths and maintained using sequence numbers.	• <i>Low Control Overhead:</i> Does not require global periodic routing advertisements. • <i>Simplicity:</i> Computationally simple. • <i>Low Memory Usage:</i> Does not require large amounts of memory. • <i>Adaptability:</i> Adapts well to changing network topologies as routes are built on demand.	• <i>High Setup Delay:</i> High connection setup delay due to route discovery. • <i>Flooding Overhead:</i> Route discovery flooding increases control overhead. • <i>Scalability:</i> Poor scalability in large networks due to flooding overhead. [4]
Zone Routing protocol (ZRP)	Hybrid	Distributed	• Uses proactive routing within the local zone for immediate delivery. • Uses reactive routing for destinations outside the zone by searching neighboring zones.	• <i>Reduced Control Overhead:</i> Reduces the control overhead for longer routes by using proactive inter-zone routing. • <i>Low Routing Delay:</i> Reduces the delays for routing within a zone by using a proactive protocol.	• <i>Not suitable for high mobility:</i> periodic updates can lead to outdated routes and frequent maintenance. • <i>Fixed Zone Radius:</i> Fixed zone radius may lead to uneven energy/resource usage in heterogeneous network. • <i>Higher Complexity:</i> More complex to implement than purely proactive or reactive protocols.

Routing Protocol	Type	Central /Distributed	Definition	Key Characteristics	Limitations
Greedy Perimeter Stateless Routing (GPSR)	Position-Based	Distributed	• Uses the positions of routers and a packet's destination to make packet forwarding decisions. [5] • Uses greedy forwarding to forward packets to nodes that are always progressively closer to the destination. • If greedy forwarding fails, switches to perimeter routing to go around obstacles.	• <i>Adaptability to Mobility</i>: Handles dynamic topologies using only local neighbor information. • <i>Scalability</i>: Efficient in large networks due to localized routing and no global tables. • <i>Low Control Overhead</i>: No route discovery or maintenance; only periodic beaconing.	• <i>Reliance on Accurate Location Data</i> • <i>Beaconing Overhead in High Mobility</i>: Frequent updates in high mobility increase overhead and may cause outdated neighbor data.
Cluster-Based Routing (CBRP)	Hierarchical	Distributed	• Divides the nodes into a number of overlapping or disjoint 2-hop- diameter clusters. • A cluster head is elected for each cluster to maintain cluster membership information. • Inter- cluster routes are discovered dynamically using the cluster membership information kept at each cluster head.	• <i>Low Control Overhead</i>: Minimizes flooding traffic during route discovery by clustering nodes into groups, where only cluster heads handle route discovery. • <i>Scalability</i>: Hierarchical clustering limits route discovery to cluster heads, improving scalability in large networks.	• <i>Clustering Overhead</i>: Cluster formation and maintenance add extra overhead. • <i>Mobility Sensitivity</i>: Frequent topology changes require re-clustering. • <i>Energy Imbalance</i>: Cluster heads may drain energy faster and become bottlenecks.
Random Walk Routing	Probabilistic	Distributed	• Forwards packets by randomly selecting the next neighbor at each step. • Continues until the destination is reached or a stopping condition occurs.	• <i>Simplicity</i>: Easy to implement with random neighbor selection. • <i>Low Control Overhead</i>: No routing tables or discovery. • <i>Robustness</i>: Adapts to topology changes without relying on network structure. • <i>Adaptability</i>: Suitable for highly dynamic or unknown networks since it can operate without prior knowledge of the network.	• <i>Inefficiency</i>: Does not guarantee shortest paths, leading to longer routes. • <i>Redundancy</i>: Packets may revisit nodes or form loops. • <i>High Latency</i>: Random forwarding increases delivery time as we are forwarding the packet to a random neighbor, not the best one. • <i>Unreliability</i>: No guarantee that packets reach the destination.

4.2 AI enabled routing protocols

In this section, we study the AI-enabled routing protocols, which use the learning power of ML algorithms for optimal route path selection based on a more accurate perception of the network topology, channel status, user behavior, traffic mobility, etc

1. *Topology predictive routing protocols*: The main feature of topology-predictive routing protocols is using ML algorithms to predict the node's motion trajectories (as an approximate of the network topology, if the communication range of nodes is known) and incorporate them into the path selection mechanism.
2. *Self-adaptive learning-based routing protocols*: Most learning-based routing protocols use Reinforcement Learning (RL) to make routing decisions by continued and online learning of the environment and their decision consequences on desired performance metrics such as delay, throughput, energy efficiency, and fairness. A key advantage of RL-based algorithms is their abstract formulation which brings independence from topology prediction and channel estimation.

Table 2: Review of AI enabled Routing protocols for UAVs

Routing Protocol	Type	Central / Distributed	Definition	Key Characteristics	Limitations
Predictive Dijkstra [6]	Topology predictive	Distributed	- Predicts the intermediate nodes locations using ML methods. - Incorporates this predictive information into the path selection criterion based on Dijkstra's shortest path algorithm.	<i>Improved Performance:</i> Outperforms standard Dijkstra's algorithm, especially at high node mobility. <i>Mobility Awareness:</i> Anticipates node movement through trajectory prediction.	<i>Prediction Dependency:</i> Reliance on accurate trajectory prediction methods. <i>Global information requirement:</i> Need for global location information exchange.
Geographic Position Mobility Oriented Routing(GPMOR) [7]	Topology predictive/ Geographic-Based	Distributed	- Uses the Gauss-Markov mobility model to predict the movement of UAVs to reduce the impact of highly dynamic movements. - Selects the next hop according to the mobility relationship and to the Euclidean distance.	<i>Improved Performance:</i> Provides effective and accurate data forwarding solutions according to the experiments at [7] <i>intermittent connectivity:</i> Decreases the impact of intermittent connectivity. <i>Reduced latency and packet loss:</i> Better latency and packet delay rate than other position based routing protocols.	<i>Prediction Dependency:</i> Reliance on accurate trajectory prediction methods. <i>Global information requirement:</i> Need for global location information exchange.
Predictive Ad-hoc Routing fueled by Reinforcement learning and Trajectory Knowledge (PARRoT)[8]	Topology predictive/ ML-powered	Distributed	- Each agent estimates its own future position based on the current position and propagates the result to other nodes. - Uses a Q-table to evaluate best next-hop neighbors, and selects the neighbor with the highest expected routing quality. - Continuously updates routes based on new data and predictions - Maintains multiple paths enabling fast adaptation to link failures.	<i>Improved Performance:</i> Higher robustness and a significantly low end-to-end latency. <i>Decoupled Networking and Path Planning Suitable</i> for dynamic environments because it separates networking from path planning layer, allowing communication to operate independently of UAV movement decisions. <i>Self-learning</i> Uses Reinforcement Learning(RL) to improve routing decisions over time.	<i>Prediction Dependency:</i> Reliance on accurate trajectory prediction methods. <i>Information Overhead:</i> Requires frequent exchange of location and prediction data. <i>Computational Complexity:</i> Additional processing due to prediction and learning mechanisms.

Routing Protocol	Type	Central /Distributed	Definition	Key Characteristics	Limitations
Q-Routing [9]	Self-adaptive	Distributed	• Variant of Q-Learning [10], which is an incremental (or asynchronous) version of dynamic programming for solving multistage decision problems. • Each communication node has a separate local controller, which does not rely on global information of the network for decision making and refinement of its routing policy. • Each node learns a routing policy which balances minimizing the number of "hops" a packet will take with the possibility of congestion along popular routes. • Continuously updates routing decisions based on packet delivery time • When a Q-value is not updated for a long time, its reliability in representing the true state of the network goes down. In Q-Routing there is no way of quantifying the reliability of a Q-value	• <i>High Load Performance:</i> Handles high network traffic efficiently by avoiding congestion. • <i>Adaptive Learning:</i> Learns efficient routing policies without prior knowledge of topology. • <i>Robustness:</i> Adapts to dynamic changes in topology, traffic patterns, and load. • <i>Decentralized Operation:</i> Does not require centralized control or global network information.	• <i>Initial Learning Overhead:</i> Poor performance during early learning phase. • <i>Low Load Inefficiency:</i> Does not outperform shortest-path routing under low traffic conditions. • <i>Storage Overhead:</i> Requires maintaining Q-tables for multiple destinations and neighbors. • <i>Optimal policies:</i> Unable to learn new optimal policies under decreasing load conditions. • <i>No Universal Optimality:</i> Does not perform best under all network load conditions.
Predictive Q-Routing (PQ-Routing) [11]	Self-adaptive	Distributed	• Extension of Q-routing that applies reinforcement learning for adaptive packet routing in dynamic networks • Stores Q-values (estimated delivery time) and keeps the best past experiences and reuses them by predicting the traffic trend. • Introduces recovery rates to estimate how quickly congested paths become efficient again • Uses controlled exploration (probing) to periodically test alternative paths • Selects routes based on predicted Q-values rather than only current estimates	• <i>Improved Performance:</i> Achieves lower packet delivery time than Q-routing after initial learning • <i>Better Congestion Handling:</i> Avoids persistent bottlenecks by alternating between multiple paths • <i>Faster Adaptation:</i> Shorter adaptation time is generally needed when the traffic patterns change than Q-routing, since the routing policy of PQ-routing depends not only on the current Q-values but also on the recovery rates. • <i>Recovery Awareness:</i> Reuses previously congested paths when they become efficient again • <i>Stable Learning:</i> Learns more stable routing policies in dynamic environments	• <i>Additional Memory Requirement:</i> Needs more memory for recovery rate estimation. • <i>Increased Complexity:</i> More complex than standard Q-routing due to prediction and probing mechanisms • <i>Inaccurate Recovery rate:</i> Not accurate in estimating the recovery rate under varying topology change rates (e.g., when nodes start to moving faster) • <i>Delays handling:</i> Only works for delays arising from the queuing congestion and not delays coming from the network topology change.

Routing Protocol	Type	Central /Distributed	Definition	Key Characteristics	Limitations
Confidence-based Dual Reinforcement Q-Routing (CDRQ-Routing) [12]	Topology predictive	Distributed	- Improves both the quality and quantity of exploration of Q-Routing. - Associates confidence values (between 0-1) with each of the Q-values in the network to represent their reliability. - Updates Q-values using both forward exploration (based on next-hop information) and backward exploration (based on previously traversed path) (whereas in Q-Routing only forward exploration is used). - Adjusts the learning rate (The amount of adaptation for a Q-value) dynamically based on the confidence of current and received Q-values (whereas in Q-Routing a fixed learning rate is used for all updates) - Uses packets to carry routing information between nodes during transmission - Makes routing decisions by selecting the neighbor with the minimum estimated delivery time based on the Q-values.	Faster Convergence: Learns optimal routing policies significantly faster than Q-routing Improved Routing Policy: learns a better routing policy more than twice as fast as Q-Routing at various load levels. Better Adaptation: Adapts effectively to varying traffic patterns and network load conditions Higher Load Tolerance: Sustains higher network load compared to shortest-path, Bellman-Ford, and Q-routing Efficient Exploration: Combines forward and backward exploration to improve learning efficiency Low Overhead: The additional overhead of adding backward exploration and C-values is less than 0.2 %	Increased Complexity: Introduces additional components (confidence values and dual updates) compared to Q-routing Additional Memory Requirement: Requires storage for Q-values and corresponding confidence values. Initial Learning Phase: Performance is suboptimal during early exploration before convergence Assumption Limitations: Performance evaluation assumes ideal conditions (e.g., infinite queues, fixed link delays)
QMR [13]	RL	Distributed	- Based on Q-learning, specifically designed for FANETs. - Multi-objective reward function: jointly considers one-hop delay and next-hop residual energy. - Adaptive learning rate: each link gets its own learning rate, dynamically adjusted based on one-hop delay variance. - Adaptive discount factor: each node's discount factor adapts according to the stability of its neighbour set. -weighted Q-value selection: first filters neighbours that can meet the packet's deadline, then picks the one with the highest product of current link quality () and long-term Q-value.	Low Delay: Speed filtering and adaptive learning reduce average end-to-end delay (up to 42% lower than QGeo). Reduced Maximum Delay: Maximum delay lowered by about 47% on average. High PDR: Adaptive discount factor, adaptive learning rate, and penalty mechanism improve packet delivery ratio (up to 10% increase). Energy Efficient: Energy consumption is explicitly part of the reward function; reduces energy by 18–19% compared to QGeo.	- Fixed weight ω in reward function: same delay-energy trade-off for all nodes regardless of their own battery level (addressed in this work by a dynamic ω). - Initial Q-values set to 0.5 $\rightarrow$ suboptimal routing during early learning phase until the network is explored. - The exploration mechanism can occasionally select a historically poor link if its current is high, causing temporary drops in PDR. - Relies on GPS and predictable mobility for position estimation; performance may degrade if position information is inaccurate or delayed.

5 Protocol to Modify : QMR

The protocol that will be modified is QMR. We chose this protocol because of its interesting reward function, which uses both the energy of the next hop and the delay of the chosen path, enabling a balance between communication performance and energy efficiency.

5.1 QMR Reinforcement learning algorithm

The following is the QMR algorithm as taken from QMR paper [13]

Algorithm 1 QMR

```
1: Phase 1: Location acquisition
2: if the time of updating node location arrives then
3:   Each node obtains its location via GPS;
4: end if

5: Phase 2: Routing neighbor discovery
6: if the time of sending HELLO packets arrives then
7:   Each node sends a HELLO packet;
8:   Receiving nodes reconfirm their neighbors and update neighbor tables according to
   HELLO packets;
9: end if

10: Phase 3: Routing decision
11: while data packets need to be sent do
12:   if the set of candidate neighbors is not empty then
13:     Make routing decision based on Q-learning;
14:   else if there are neighbors whose actual velocity is greater than 0 then
15:     Select the neighbor associated with the maximum actual velocity;
16:   else
17:     Apply penalty mechanism;
18:   end if
19: end while

20: Phase 4: Q-learning
21: if no information about neighbors exists then
22:   Initialize the Q-value of each neighbor;
23: end if
24: Select the neighbor associated with the largest  $\kappa$ -weighted Q-value;
25: if a neighbor is selected to forward data packets then
26:   Update the neighbor's Q-value using a reward;
27: end if

28: Phase 5: Penalty mechanism
29: if a routing hole problem is encountered, or forwarding nodes do not receive ACK packets
   then
30:   Give minimum reward to related neighbors;
31:   Update Q-values of related neighbors;
32: end if
```

5.2 Algorithm steps

Phase 1: Location Acquisition

- Each drone obtains its own geographic coordinates through GPS.
- The ground station's position is assumed to be known to all drones (e.g., as a mission parameter). The drone later learns the positions of the other drones during phase 2.

Phase 2: Routing Neighbor Discovery

- Every node periodically sends HELLO packets with their own parameters. These packets contain:
 - Position
 - Residual energy (E_j)
 - Mobility info (speed, heading)
 - Queuing delay, discount factor, etc.
- Upon receiving a HELLO, a node updates its neighbor table with the advertised values, thereby building a local view of the one-hop neighborhood.

Phase 3: Routing Decision

- Before selecting a next-hop, drone A performs a **deadline-aware speed check** (in Fig. 2):
 - It updates the packet's remaining deadline.
 - It computes the **required velocity**: the minimum progress speed needed to reach the depot before the deadline expires.
 - For each neighbor, it estimates the **actual velocity**: the reduction in distance to the depot per unit time, based on the neighbor's predicted position and the one-hop delay.
- Neighbors whose actual velocity is at least the required velocity become **candidate forwarders** and proceed to the Q-learning decision (Phase 4).
 - **Example in Fig. 2:** Both B and C satisfy the speed requirement, so they both enter the candidate set.
- If no candidate meets the deadline, the drone selects the neighbor with the highest positive actual velocity (fallback). When all actual velocities are non-positive, the penalty mechanism (Phase 5) is triggered.

Phase 4: Q-Learning Decision (κ -Weighted Q-Value)

- When a data packet reaches node i and no prior knowledge about the available neighbors exists, the Q-values of all one-hop neighbors are initialized to a default value ($Q = 0.5$).
- The forwarding decision is made by selecting the neighbor j that maximizes the κ -weighted Q-value:

$$j^* = \arg \max_j (\kappa_{i,j} \cdot Q(d, j)),$$

where

$$\kappa_{i,j} = M_{i,j} \times LQ_{i,j}$$

combines the current link intimacy $M_{i,j}$ ¹ and the measured link quality $LQ_{i,j}$ ², while $Q(d, j)$ is the learned long-term quality of using neighbor j to reach destination d .

¹How close the neighbour will be when the packet actually arrives (i.e., the predicted future distance)

²The bidirectional reliability of the wireless link, i.e., how likely a packet and its acknowledgement will be successfully exchanged

- To avoid permanent lock-in to suboptimal routes, a 10% ε -greedy exploration step is employed: with probability $\varepsilon = 0.1$, the drone selects a random neighbor from its one-hop neighbor set instead of the κ -weighted best choice
- Once the forwarding node receives the acknowledgment from the chosen neighbor, it computes the immediate reward and updates the corresponding Q-value; this makes the neighbor more likely to be selected again if the reward was high, or less likely if it was low.
- This process repeats for every hop until the packet reaches the ground station or is dropped due to a routing failure.

Phase 5: Penalty Mechanism

- The penalty mechanism is triggered in two cases:
 - a **routing hole** is encountered (no neighbor can move the packet closer to the destination),
 - the forwarding node does **not receive an ACK** from the next-hop neighbor, indicating a transmission failure.
- In either case, the affected neighbor is given the **minimum possible reward**, and its Q-value is updated accordingly.
- This negative feedback discourages the use of unreliable or dead-end links in future routing decisions, improving overall path stability.

6 Protocol Modification: Modifying the Reward function

The key alteration in our approach lies in the reward function, where the static trade-off weight ω is replaced by a dynamic value that responds to the energy state of the forwarding drone itself.

6.1 Reward function

The forwarding decision is made by maximizing the κ -weighted Q-value, i.e., the product of the current link quality κ and the learned Q-value. The Q-values themselves are updated after each forwarding action using a reward function that combines the one-hop delay with the residual energy of the chosen neighbor:

$$R = \omega \cdot e^{-d_{i,j}} + (1 - \omega) \cdot E_j,$$

where $d_{i,j}$ is the estimated one-hop delay from node i to neighbor j , obtained from the smoothed medium access ³, and queuing delays ⁴ of recent packets sent by i to j . E_j is the remaining energy fraction of neighbor j .

The weight ω is fixed to 0.8 in the original protocol; therefore, **80 %** of the reward depends on the delay and only **20%** on the neighbor's energy. As a result, the protocol always favors the fastest link, even when that neighbor has very low energy, and it never adapts its preference to the energy state of the forwarding drone itself.

³The time the sender's radio spends contending for the channel and transmitting the packet to the specific neighbor

⁴The time the packet waited in the sender's transmission queue before reaching the head of the queue

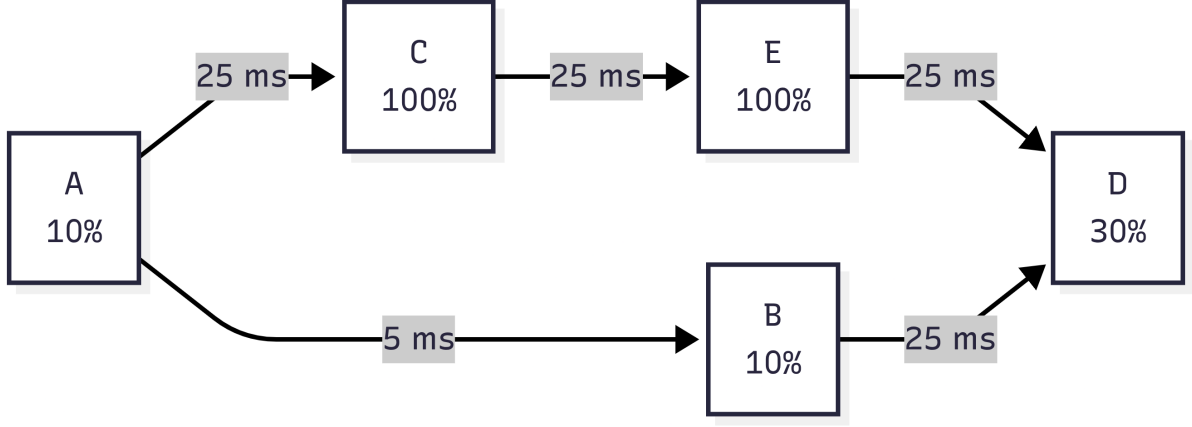


Figure 2: Illustration Setup

Illustrative example: For a concrete illustration, consider the topology in Fig. 2. Assume, for simplicity, that 1% of battery capacity sustains one minute of operation, so the lifetime of a multi-hop path is dictated by the node with the smallest residual energy: the bottleneck energy, because the route fails as soon as its weakest node exhausts its battery. Two routes are available from source A to destination D:

1. $A \rightarrow B \rightarrow D$: bottleneck energy 10% $\rightarrow$ lifetime 10 min
2. $A \rightarrow C \rightarrow E \rightarrow D$: bottleneck energy 30% $\rightarrow$ lifetime 30 min

The initial forwarding decision is influenced by the current link intimacy κ (which favors closer neighbors). However, the Q-values are updated after each hop using the reward function. With the fixed weight $\omega = 0.8$, the reward places far more emphasis on delay than on energy. Over successive packets, this causes the Q-value of the faster (but energy-poor) route to dominate, and the algorithm persistently selects $A \rightarrow B \rightarrow D$ even though it offers only one-third of the lifetime of the alternative $A \rightarrow C \rightarrow E \rightarrow D$. This illustrates how the original fixed trade-off sacrifices network longevity for marginal reductions in latency.

6.1.1 First Protocol Optimization: Dynamic ω

We change ω to a dynamic value :

- If the current node has a low battery, it should prioritize getting the packet out quickly (higher ω) to minimize its own transmission energy drain and avoid holding packets, because a higher delay means the packet sits longer in the node's transmission queue.
- If the current node has a high battery, it can afford to be more energy-conscious about the entire path (lower ω), favoring neighbors with higher residual energy even if they're slightly slower.

7 Experiments

To evaluate the impact of the dynamic weight, we compare the original fixed- ω protocol with our modified version. The QMR algorithm was originally implemented in C for the event-driven wireless network simulator WSN⁵, which is no longer maintained. We, therefore, base our experiments on a publicly available Python reimplement⁶ developed for the DroNet simu-

⁵<http://wsnet.gforge.inria.fr/>

⁶<https://github.com/LeonardoBerti00/QMR-Q-learning-based-Multi-objective-optimization-Routing-protocol>

Table 3: Dynamic Adjustment of Delay Weight Based on Battery Level

Current Node Battery	ω (Delay Weight)	Behavior
Low ($< 50\%$)	High (0.8)	speed-oriented, but still leaves 20% weight to avoid dying neighbors.
High ($\geq 50\%$)	Low (0.3)	Now 70% weight on neighbor energy, rarely picks a dying node and saves the network’s life-time.

lator⁷. Because we do not use the full DroNet mobility framework, we extracted only the core QMR routing logic from that repository and applied several corrections to make it faithful to the published protocol (see Paragraph 7.1). Our implementation is publicly available online.⁸

7.1 Initial Implementation Corrections and Adaptations

Corrections of the Original Python Implementation The main adjustments are summarized below:

- **Reward function correction:** The python code computed the reward as $\omega \cdot e^{+\text{delay}} + (1 - \omega) \cdot (\text{own energy})$. This was changed to

$$\omega \cdot e^{-\text{delay} / \text{max_delay}} + (1 - \omega) \cdot E_j,$$

where E_j is the residual energy fraction of the candidate next-hop node, and the delay is normalized by a running maximum `max_delay`. This adjustment aligns the reward with Equation (3) of the paper and with the official C implementation.

- **Delay normalization:** A variable `max_delay` (equivalent to `nb_maxdelay` in the C code) stores the largest one-hop delay ever observed. It is updated inside `computeReward` and used to scale the raw delay before exponentiation, preventing floating-point underflow and ensuring a stable reward signal across diverse network conditions.
- **Neighbor energy access:** In the python implementation, `feedback` called `self.computeReward(outcome, delay)` without passing the neighbor identifier. Consequently, the reward function could not retrieve the energy of the intended next-hop node and instead used the forwarding drone’s own energy. The call has been corrected to `self.computeReward(delay, id_j)`, allowing the function to access the actual residual energy of the selected neighbor.
- **Baseline weight and exploration:** A fixed weight $\omega = 0.8$ is employed, consistent with the paper’s parameter setting. The ε -greedy exploration mechanism (10% random neighbor selection) is added to match the original protocol.

Adaptations for the Static Environment Setup The following simplifications are applied because the evaluation is performed on static topologies where the UAV positions remain fixed:

- **Link quality:** The link quality is set to $LQ = 1$ for all links, assuming perfect bidirectional communication, since there is no interference or packet loss.
- **One-hop delay:** MAC delay and queuing delay are set once at topology creation and remain constant throughout the simulation since the nodes do not move.

⁷<https://github.com/Andrea94c/DroNETworkSimulator>

⁸<https://github.com/nermine11/QMR-Q-learning-based-Multi-objective-optimization-Routing-protocol>

- **Packet deadline:** The initial deadline is made arbitrarily large so that packets never expire; the speed filter only discards neighbors that would move the packet away from the destination. This design choice removes real-time constraints from the experiment, allowing us to evaluate the energy-aware routing behavior of the modified protocol in isolation.
- **Learning parameters:** The learning rate⁹ α and the discount factor¹⁰ γ are fixed at 0.3 and 0.9. In a static network, delays never change and neighbors never move, so the original adaptive formulas cannot operate.

7.2 First Experiment : Paths Comparison

We run our first experiment to compare the paths taken using the original code and our optimized code. The experiment parameters are as follows:

- 25 drones
- Source: drone 3
- $500 \times 500 \text{ m}^2$ area; ground station at the center
- Communication range 200 m; edges between drones with a distance of $< 200 \text{ m}$.
- Random initial energies 5%–100% of the maximum battery capacity.
- CBR source rate 200 ms; 10 packets per source.
- Energy drain: idle 10%/min, transmission 15% per packet.
- Path lifetime = bottleneck energy (lowest remaining battery on path) $\div 10 \text{ min}$.
- Fixed seed 46: both fixed ($\omega = 0.8$) and dynamic ($\omega = 0.8/0.3$) run on identical topologies.

7.2.1 Start of the experiment: Why Fixed and Dynamic ω Initially Behave Identically

We initially compared the original fixed ω policy with the proposed dynamic- ω modification on a small illustrative topology to see a difference in the selected paths. However, both policies consistently chose the exact same routes over 1000 packets. This behavior stems from the initialization of the Q-table: **all Q-values are set to the same default (0.5)**. In the first forwarding decision, multiple neighbors therefore share an identical κ -weighted Q-value, and the tie is broken by the link intimacy factor κ , which favors the closest drone. That drone is then rewarded, its Q-value increases, and it becomes the unique best choice for all subsequent packets. Consequently, the protocol locks onto the initially selected neighbor and never explores alternative, potentially more energy-efficient paths. The 10% ϵ -greedy exploration can eventually break this lock, but with a limited number of packets exploration may not occur at all, or may occur too late to allow the alternative, energy-efficient path to be discovered and reinforced. Consequently, the dynamic ω , despite its potential to favor energy-rich routes, never gets an opportunity to influence the routing decision.

⁹The learning rate α controls how much a new experience overrides the old Q-value. In QMR, α is adapted per link based on the variance of the one-hop delay; a stable link (low variance) gets a small α , an unstable one a large α . In our static network, delays are constant, so the variance is zero and the formula always returns the same value.

¹⁰The discount factor γ determines how much the agent trusts future rewards reported by a neighbour. QMR adjusts γ by comparing the neighbour set at two successive time intervals; a stable set yields a high γ , a rapidly changing set a low γ . With static nodes, the neighbour set never changes, so γ cannot adapt.

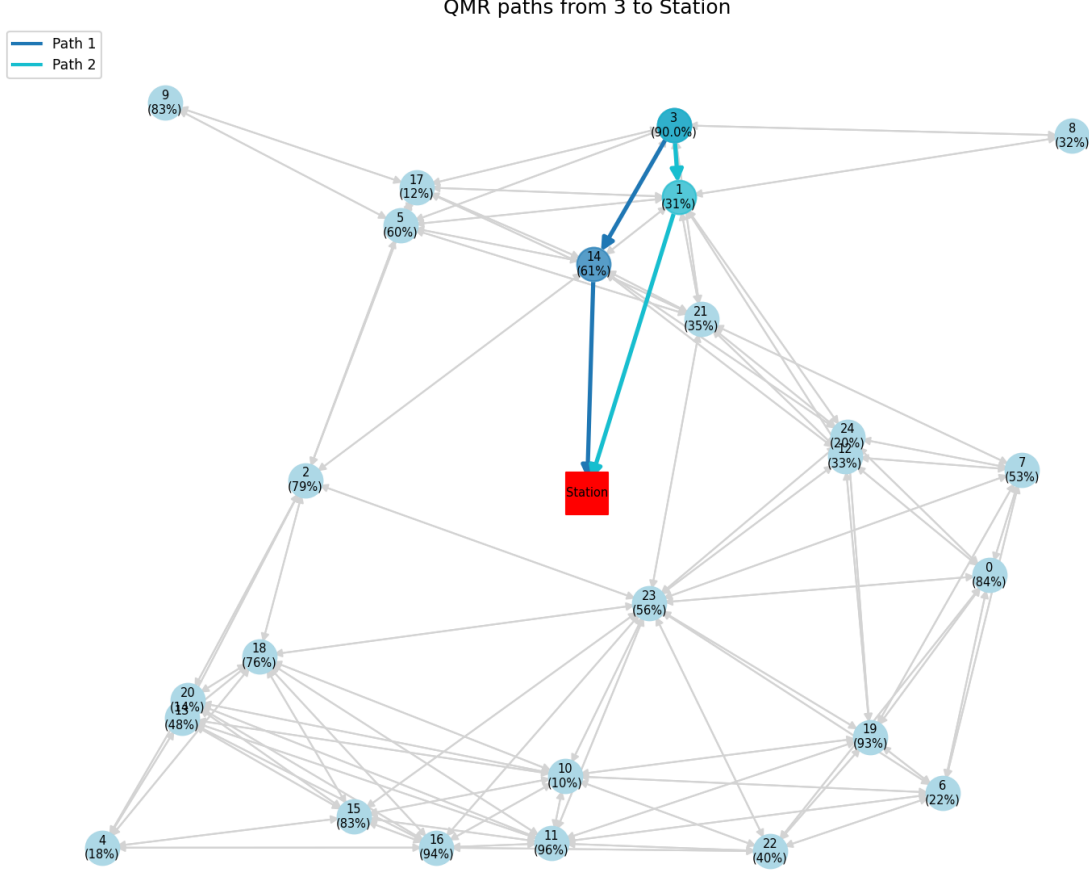


Figure 3: Original protocol paths visualization

7.2.2 Second protocol optimization: Energy-Based Initial Q-Values

To allow the dynamic ω to influence routing from the very first packet, we replace the default uniform initialization of the Q-table with a value that reflects the neighbor's current energy. Specifically, for each neighbor j , the initial Q-value is set to

$$Q_{\text{init}}(d, j) = \frac{\text{residual_energy}_j}{100},$$

i.e., the neighbor's remaining battery expressed as a fraction of its maximum capacity (a number between 0 and 1). As a result, neighbors with higher residual energy receive higher initial Q-values, allowing the protocol to prioritize more energy-efficient routes from the start.

7.2.3 Results

Original Protocol In Fig. 3 We send 10 packets from drone 3 to the ground station.

As we can see in the debug output below, Drone 3 has six possible next-hop neighbors (nodes 1, 5, 8, 14, 17, and 21). For each neighbor, the required velocity v_{req} and the actual velocity v_{act} are computed. If $v_{\text{act}} \leq 0$, the neighbor is discarded; this is why node 8 is immediately eliminated ($v_{\text{act}} = -4.040$). Among the remaining candidates, the protocol computes $\kappa = M \times LQ$ and selects the neighbor with the highest κ -weighted Q-value.

Since all Q-values start at 0.5, the neighbor with the largest κ , which is node 1 ($\kappa = 0.729$) should be chosen. However, the 10% ε -greedy exploration step overrides this decision and randomly

picks node 14. The packet, therefore, follows the route $3 \rightarrow 14 \rightarrow \text{Station}$.

At the moment of forwarding, the residual energies¹¹ of the traversed drones are 18.97% (drone 3) and 60.97% (drone 14). The bottleneck energy is the smaller of the two, i.e., 18.97%. Using the relation that 10% battery sustains one minute of communication, the lifetime of this path is

$$\frac{18.97}{10} \approx 1.90 \text{ minutes.}$$

[DEBUG] Drone 3 selecting next hop

My residual energy: 74.97%

My position: (291.86524278680304, 473.59976098836455), Depot: (250.0, 250.0)

Neighbor 1: residual_energy=30.97%, dist_ij=54.2, actual_v=12.373, req_v=0.114

-> candidate: M=0.729, LQ=1.000, k=0.729

Neighbor 5: residual_energy=59.97%, dist_ij=158.6, actual_v=3.105, req_v=0.114

-> candidate: M=0.207, LQ=1.000, k=0.207

Neighbor 8: residual_energy=31.97%, dist_ij=182.8, actual_v=-4.040, req_v=0.114

-> discarded (actual_v <= 0)

Neighbor 14: residual_energy=60.97%, dist_ij=106.1, actual_v=10.572, req_v=0.114

-> candidate: M=0.469, LQ=1.000, k=0.469

Neighbor 17: residual_energy=11.97%, dist_ij=142.3, actual_v=2.226, req_v=0.114

-> candidate: M=0.289, LQ=1.000, k=0.289

Neighbor 21: residual_energy=34.97%, dist_ij=127.8, actual_v=10.257, req_v=0.114

-> candidate: M=0.361, LQ=1.000, k=0.361

Candidate scores:

1: Q=0.500, k=0.729, weighted=0.364

5: Q=0.500, k=0.207, weighted=0.103

14: Q=0.500, k=0.469, weighted=0.235

17: Q=0.500, k=0.289, weighted=0.144

21: Q=0.500, k=0.361, weighted=0.181

\(\backslash\text{varepsilon}\)-greedy override: was 1, now randomly picked 14

Final choice: 14

For the second packet (see debug output below), node 1 is chosen because its κ -weighted Q-value (0.364) exceeds that of node 14 (0.297), even though node 14's Q-value has risen to 0.633 while node 1's remains at 0.5. The larger intimacy factor κ of node 1 (0.729 vs. 0.469) outweighs the higher learned Q-value of node 14, demonstrating that a larger κ (closer proximity) can outweigh a higher Q-value, thus prioritizing immediate link intimacy over accumulated experience. The chosen path is therefore $3 \rightarrow 1 \rightarrow \text{Station}$. The residual energies at the moment of forwarding are 59.93% (drone 3) and 30.93% (drone 1), giving a bottleneck energy of 30.93% and a packet lifetime of

$$\frac{30.93}{10} \approx 3.09 \text{ minutes.}$$

If node 14 had been selected instead of node 1, the path would have been $3 \rightarrow 14 \rightarrow \text{Station}$. At that moment, node 14 had 45.93% residual energy, yielding a bottleneck of 45.93% and a lifetime of

$$\frac{45.93}{10} \approx 4.59 \text{ minutes.}$$

This is roughly 50% higher than the 3.09 minutes obtained by following the κ -preferred route, illustrating how the fixed weight $\omega = 0.8$ can lead the protocol to sacrifice path longevity for immediate link intimacy.

¹¹Remaining battery percentage

```

[DEBUG] Drone 3 selecting next hop
My residual energy: 59.93%
My position: (291.86524278680304, 473.59976098836455), Depot: (250.0, 250.0)
Neighbor 1: residual_energy=30.93%, dist_ij=54.2, actual_v=12.373, req_v=0.114
-> candidate: M=0.729, LQ=1.000, k=0.729
Neighbor 5: residual_energy=59.93%, dist_ij=158.6, actual_v=3.105, req_v=0.114
-> candidate: M=0.207, LQ=1.000, k=0.207
Neighbor 8: residual_energy=31.93%, dist_ij=182.8, actual_v=-4.040, req_v=0.114
-> discarded (actual_v <= 0)
Neighbor 14: residual_energy=45.93%, dist_ij=106.1, actual_v=10.572, req_v=0.114
-> candidate: M=0.469, LQ=1.000, k=0.469
Neighbor 17: residual_energy=11.93%, dist_ij=142.3, actual_v=2.226, req_v=0.114
-> candidate: M=0.289, LQ=1.000, k=0.289
Neighbor 21: residual_energy=34.93%, dist_ij=127.8, actual_v=10.257, req_v=0.114
-> candidate: M=0.361, LQ=1.000, k=0.361
Candidate scores:
1: Q=0.500, k=0.729, weighted=0.364
5: Q=0.500, k=0.207, weighted=0.103
14: Q=0.633, k=0.469, weighted=0.297
17: Q=0.500, k=0.289, weighted=0.144
21: Q=0.500, k=0.361, weighted=0.181
Final choice: 1

```

Over the subsequent packets, the chosen path remains consistently $3 \rightarrow 1 \rightarrow \text{Station}$. As node 1 continues to receive high rewards due to its low delay, its Q-value increases further, widening the gap with other candidates. Even though the ε -greedy exploration occasionally tries alternative neighbors, the dominance of node 1's κ -weighted score, driven by its large intimacy factor and high reward due to its low delay, prevents any other route from being selected. Consequently, the protocol locks onto the fast but energy-poor relay, regardless of the energy cost.

Optimized Protocol In Fig. 4, we send 10 packets from drone 3 to the ground station. As we can see in the debug output below, with the energy-based initial Q-values, node 14 starts with a higher Q (0.610) than node 1 (0.310) since its residual energy is higher. In the very first packet (source battery 74.97%), the κ -weighted score of node 14 (0.286) already exceeds that of node 1 (0.226), so we select node 14 as the next hop. The path taken is therefore $3 \rightarrow 14 \rightarrow \text{Station}$. Node 14's Q-value is then reinforced, rising to 0.883 after the first update, an increase of 0.273 (+45%). This strong growth is a direct result of the dynamic weight $\omega = 0.3$, which gives 70% of the reward to the neighbor's residual energy. In contrast, under fixed $\omega = 0.8$ (see Paragraph 7.2.3), the same neighbor 14 is selected once by the ε -greedy exploration and its Q-value improves only from 0.5 to 0.633, an increase of just 0.133 (+27%). Thus, the dynamic policy provides nearly twice the Q-value gain for energy-rich neighbors, causing the protocol to quickly converge on the energy-efficient route.

```

[DEBUG] Drone 3 selecting next hop
My residual energy: 74.97%
My position: (291.86524278680304, 473.59976098836455), Depot: (250.0, 250.0)
Neighbor 1: residual_energy=30.97%, dist_ij=54.2, actual_v=12.373, req_v=0.114
-> candidate: M=0.729, LQ=1.000, k=0.729
Neighbor 5: residual_energy=59.97%, dist_ij=158.6, actual_v=3.105, req_v=0.114
-> candidate: M=0.207, LQ=1.000, k=0.207
Neighbor 8: residual_energy=31.97%, dist_ij=182.8, actual_v=-4.040, req_v=0.114

```

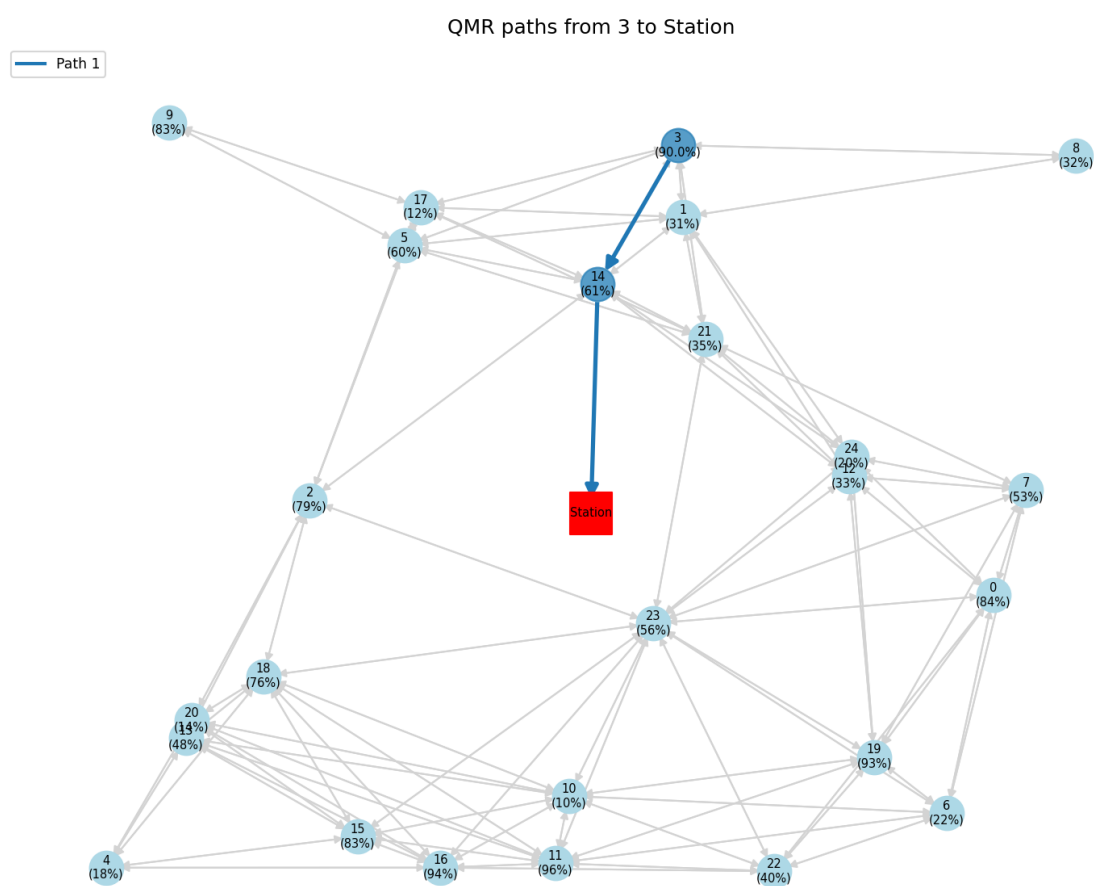


Figure 4: Optimized protocol paths visualization

```

-> discarded (actual_v <= 0)
Neighbor 14: residual_energy=60.97%, dist_ij=106.1, actual_v=10.572, req_v=0.114
-> candidate: M=0.469, LQ=1.000, k=0.469
Neighbor 17: residual_energy=11.97%, dist_ij=142.3, actual_v=2.226, req_v=0.114
-> candidate: M=0.289, LQ=1.000, k=0.289
Neighbor 21: residual_energy=34.97%, dist_ij=127.8, actual_v=10.257, req_v=0.114
-> candidate: M=0.361, LQ=1.000, k=0.361
Candidate scores:
1: Q=0.310, k=0.729, weighted=0.226
5: Q=0.600, k=0.207, weighted=0.124
14: Q=0.610, k=0.469, weighted=0.286
17: Q=0.120, k=0.289, weighted=0.035
21: Q=0.350, k=0.361, weighted=0.126
Final choice: 14

```

Over the next packet (see debug output below), we choose also the node 14 as our next hop because its κ -weighted Q-value is the highest among candidate neighbors. So the chosen path again is $3 \rightarrow 14 \rightarrow \text{Station}$. the bottleneck energy of this path is the energy of node 14: 45.93%, giving a path lifetime of

$$\frac{45.93}{10} \approx 4.59 \text{ minutes.}$$

For comparison, the fixed policy chose node 1 in the same packet, yielding a lifetime of only 3.09 minutes, **so we achieved with our optimizations a 49% improvement.**

```

[DEBUG] Drone 3 selecting next hop
My residual energy: 59.93%
My position: (291.86524278680304, 473.59976098836455), Depot: (250.0, 250.0)
Neighbor 1: residual_energy=30.93%, dist_ij=54.2, actual_v=12.373, req_v=0.114
-> candidate: M=0.729, LQ=1.000, k=0.729
Neighbor 5: residual_energy=59.93%, dist_ij=158.6, actual_v=3.105, req_v=0.114
-> candidate: M=0.207, LQ=1.000, k=0.207
Neighbor 8: residual_energy=31.93%, dist_ij=182.8, actual_v=-4.040, req_v=0.114
-> discarded (actual_v <= 0)
Neighbor 14: residual_energy=45.93%, dist_ij=106.1, actual_v=10.572, req_v=0.114
-> candidate: M=0.469, LQ=1.000, k=0.469
Neighbor 17: residual_energy=11.93%, dist_ij=142.3, actual_v=2.226, req_v=0.114
-> candidate: M=0.289, LQ=1.000, k=0.289
Neighbor 21: residual_energy=34.93%, dist_ij=127.8, actual_v=10.257, req_v=0.114
-> candidate: M=0.361, LQ=1.000, k=0.361
Candidate scores:
1: Q=0.310, k=0.729, weighted=0.226
5: Q=0.600, k=0.207, weighted=0.124
14: Q=0.913, k=0.469, weighted=0.429
17: Q=0.120, k=0.289, weighted=0.035
21: Q=0.350, k=0.361, weighted=0.126
Final choice: 14

```

Over the subsequent packets, the chosen path remains consistently $3 \rightarrow 14 \rightarrow \text{Station}$. Node 14's Q-value, already high due to the energy-biased reward, is further increased after each successful transmission, widening the gap with node 1. The protocol, therefore, locks onto the energy rich relay, maintaining a higher bottleneck energy and a longer path lifetime.

7.3 Second Experiment: Network simulations

To evaluate the impact of the dynamic weight on network lifetime, we conduct a series of static network simulations. The key parameters are as follows:

- **Network topology:** A variable number of UAVs (from 1 to 49) are randomly deployed in a $500 \times 500 \text{ m}^2$ area, along with a single ground station positioned at the center. The ground station acts solely as the destination node; it has unlimited energy and does not relay packets. UAV positions are generated such that any two drones are at least 10 m apart, and the resulting communication graph (based on a transmission range of 200 m) is guaranteed to be connected. The topology is a geometric random graph: an edge exists between any two nodes whose Euclidean distance is strictly less than the communication range.
- **Energy model:** Each UAV is initialized with a random battery level between 5% and 100% of its maximum capacity. Two energy consumption models are considered:
 - *Idle drain:* all drones continuously consume 10% of their maximum battery per minute of operation.
 - *Transmission cost:* every successful packet transmission (by the source or a relay) instantly depletes 15% of the sender’s remaining battery.
- **Simulation procedure:** For each given number of UAVs N , exactly 50 independent random topologies are generated. In every topology, each drone in turn acts as the source and sends 1000 packets to the ground station. For every successfully delivered packet, we record the **bottleneck energy** of the path it traversed, defined as the smallest residual energy among all drones on that path (excluding the station). The source generates Constant Bit Rate (CBR) traffic with a 200 ms interval between consecutive packets. This bottleneck energy is then converted into a path lifetime using the simple relation that 10% battery capacity sustains 1 minute of communication. The source’s overall performance is taken as the average of these per-packet lifetimes; failures (packets that never reach the station) are ignored in the average.
- **Comparison:** The whole process is executed twice: once with the original fixed weight $\omega = 0.8$, and once with the proposed dynamic weight that adapts based on the forwarding drone’s own battery level ($\omega = 0.3$ when battery $\geq 50\%$, $\omega = 0.8$ otherwise). The two resulting curves of average bottleneck lifetime versus number of drones are then compared.

7.3.1 Experiment Parameters

- Number of UAVs $N = 1, 2, \dots, 49$
- $500 \times 500 \text{ m}^2$ area; ground station at center
- Communication range 200 m; edges between drones with distance $< 200 \text{ m}$.
- Random initial energies 5%–100% of the maximum battery capacity..
- CBR source rate 200 ms; 1000 packets per source.
- Energy drain: idle 10%/min, transmission 15% per packet.
- Path lifetime = bottleneck energy (lowest remaining battery on path) $\div 10 \text{ min}$.
- Fixed seed 42; both fixed ($\omega = 0.8$) and dynamic ($\omega = 0.8/0.3$) run on identical topologies.

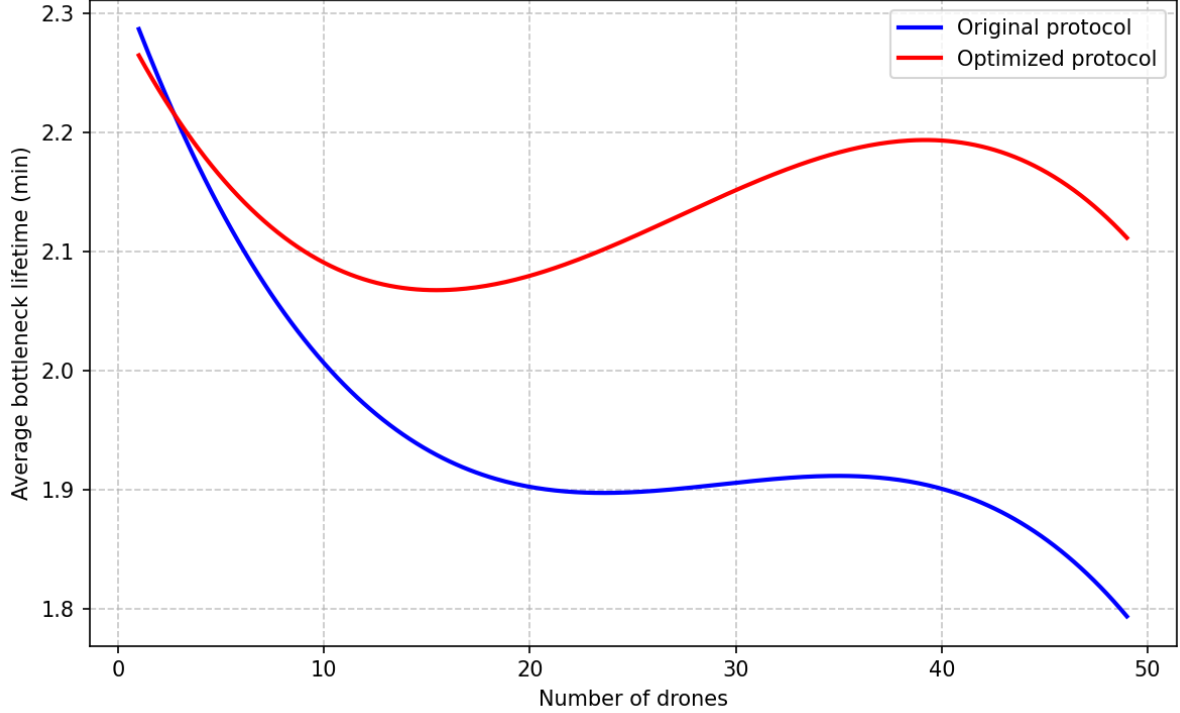


Figure 5: Average bottleneck lifetime as a function of the number of drones

7.3.2 Results

Figure 5 plots the average bottleneck lifetime (in minutes) for both the original and optimized QMR protocols as the number of UAVs increases from 1 to 49. For small networks ($N \leq 4$) the two versions produce identical results because there are too few alternative routes for the energy-aware weight to make a difference; the source often communicates directly with the ground station. From $N = 5$ onward, the optimized protocol provides a higher average lifetime at every tested node count, with a typical improvement of 0.1–0.3 minutes. The gain is most evident at intermediate densities ($10 \leq N \leq 40$), where multiple routing options exist and the protocol modifications can effectively discriminate between energy-poor and energy-rich relays. At higher densities ($40 \leq N \leq 49$), the improvement narrows slightly as the network becomes over-connected and the bottleneck energy is primarily determined by the source’s own battery depletion rather than by the choice of relay. In summary, the optimized protocol consistently achieves a higher average bottleneck path lifetime than the original for networks with at least five drones. Both the dynamic ω and the energy-based initial Q-values contribute to this improvement, confirming that the battery-responsive reward weight, combined with a well-chosen initialization, yields a consistent gain in bottleneck path lifetime and effectively extends network longevity.

8 Perspectives

The results obtained so far are encouraging, yet they are not conclusive, as they were collected on static topologies under simplified conditions. To better evaluate and extend the optimized protocol, future work can pursue the following directions:

- **Mobile network simulations:** Repeating the experiments with moving drones and realistic mobility models (e.g., random waypoint and mission-driven paths) to assess whether

the energy-based initial Q-values and dynamic ω advantages persist under frequent topology changes.

- **Real-time constraints:** Introducing tight packet deadlines to test the optimizations under strict latency constraints.
- **Adaptive Q-learning parameters:** Extending the adaptive mechanisms to the learning rate α and discount factor γ to fully exploit the self-tuning capabilities proposed in the original QMR paper.
- **Larger-scale and more diverse topologies:** Scaling up the experiments to more drones and more random topologies for stronger statistical significance.
- **Refined energy model:** Incorporating a more realistic energy consumption profile that accounts for packet reception and idle listening, to better capture the actual battery drain of a UAV.
- **Experimentation with varied parameters:** Running the same experiments with different CBR frequencies, area sizes, node counts, and packet counts to test the robustness of the optimizations.
- **Real-world deployment:** Deploying the modified protocol on real UAVs to assess its behavior in a real-world environment.

9 Conclusion

FANET routing protocols must balance communication performance and energy consumption to maintain network connectivity over time. Reinforcement learning based approaches are particularly suitable because they can adapt routing decisions to changing network conditions. This paper focused on QMR; a multi-objective Q-learning routing protocol for FANETs that jointly considers delay and energy consumption. We identified two limitations of the original protocol: a fixed reward weight that systematically favors delay over energy consumption and a uniform Q-table initialization that ignores energy information during the early learning phase.

To address these issues, this paper introduced two lightweight enhancements: a dynamic ω that adapts to the remaining battery, and an energy-based initialization of the Q-table to make the routing decision truly energy-aware.

The modified protocol was evaluated on static random topologies, and the results are encouraging: the optimized version consistently improves the average bottleneck path lifetime compared to the original protocol. Nevertheless, further experiments under mobile conditions, with realistic deadlines and on actual UAV platforms, are needed to fully validate the approach in operational FANETs.

References

- [1] N. Beijar, “Zone routing protocol (zrp),” 2002. [Online]. Available: <https://api.semanticscholar.org/CorpusID:14249841>
- [2] A. Rovira-Sugranes, F. Afghah, J. Chakareski, and A. Razi, “A review of ai-enabled routing protocols for UAV networks: Trends, challenges, and future outlook,” *CoRR*, vol. abs/2104.01283, 2021. [Online]. Available: <https://arxiv.org/abs/2104.01283>
- [3] D. B. Johnson and D. A. Maltz, *Dynamic Source Routing in Ad Hoc Wireless Networks*. Boston, MA: Springer US, 1996, pp. 153–181. [Online]. Available: https://doi.org/10.1007/978-0-585-29603-6_5
- [4] L. Hassnawi, R. Ahmad, A. Yahya, S. Aljunid, and M. Elshaikh, “Performance analysis of various routing protocols for motorway surveillance system cameras’ network,” *International Journal of Computer Science Issues (IJCSI)*, vol. 9, p. 03 2012.
- [5] B. Karp and H. T. Kung, “Gpsr: greedy perimeter stateless routing for wireless networks,” in *Proceedings of the 6th Annual International Conference on Mobile Computing and Networking*, ser. MobiCom ’00. New York, NY, USA: Association for Computing Machinery, 2000, p. 243–254. [Online]. Available: <https://doi.org/10.1145/345910.345953>
- [6] A. Rovira-Sugranes and A. Razi, “Predictive routing for dynamic uav networks,” *2017 IEEE International Conference on Wireless for Space and Extreme Environments (WiSEE)*, pp. 43–47, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:25251991>
- [7] L. Lin, Q. Sun, J. Li, and F. Yang, “A novel geographic position mobility oriented routing strategy for uavs,” vol. 8, pp. 709–716, 02 2012.
- [8] B. Sliwa, C. Schöler, M. Patchou, and C. Wietfeld, “Parrot: Predictive ad-hoc routing fueled by reinforcement learning and trajectory knowledge,” in *2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*, 2021, pp. 1–7.
- [9] J. A. Boyan and M. L. Littman, “Packet routing in dynamically changing networks: a reinforcement learning approach,” in *Proceedings of the 7th International Conference on Neural Information Processing Systems*, ser. NIPS’93. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1993, p. 671–678.
- [10] C. J. C. H. Watkins, “Learning from delayed rewards,” Ph.D. dissertation, King’s College, Oxford, 1989.
- [11] S. P. M. Choi and D.-Y. Yeung, “Predictive q-routing: a memory-based reinforcement learning approach to adaptive traffic control,” in *Proceedings of the 9th International Conference on Neural Information Processing Systems*, ser. NIPS’95. Cambridge, MA, USA: MIT Press, 1995, p. 945–951.
- [12] S. Kumar and R. Miikkulainen, “Confidence based dual reinforcement q-routing: An adaptive online network routing algorithm,” in *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence*, ser. IJCAI ’99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, p. 758–763.
- [13] J. Liu, Q. Wang, C. He, K. Jaffrès-Runser, Y. Xu, Z. Li, and Y. Xu, “Qmr: Q-learning based multi-objective optimization routing protocol for flying ad hoc networks,” *Comput. Commun.*, vol. 150, pp. 304–316, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:211080803>